Introduction & Motivation

The motivation for this study stems from a problem I first encountered during my 1st year as a Computer Science student in a Discrete Mathematics course. Our professor often challenged the class with pop quizzes and puzzles for extra credit. A few weeks before the final exam, he presented a problem that seemed simple at first glance but turned out to be quite deep:

> **"Suppose there are *n* students in a class, where *n* is divisible by 3. In each quiz, we form groups of exactly 3 students. How many quizzes can be conducted in total such that no student ever appears in a group with the same classmate more than once?"**

At the time, I was able to provide a partial solution, enough to earn the bonus marks, but the problem stuck with me. Now, in my 3rd year and preparing to serve as a teaching assistant for an algorithms course, I have returned to this puzzle with a stronger technical foundation.

Revisiting this problem made me realize that it is more than just a classroom exercise; it represents a frequent challenge in real world resource allocation and social engineering. In any collaborative environment, whether a classroom or a corporate team, there is often a goal to maximize unique interactions. By ensuring that students or employees work with as many different peers as possible, we can improve learning outcomes, ensure fairness, and strengthen team dynamics. Similar principles apply to tournament scheduling, network routing, and experimental design, where avoiding repeated pairings is essential to minimize bias and maximize efficiency.

In this paper, I study this problem from an algorithmic perspective. My goal is to determine how to construct these groupings efficiently, establish the theoretical maximum number of quizzes possible, and develop a framework for ensuring that no pair of students is ever repeated.

Problem Definition

We now formally define the problem and place it within its theoretical context.

Let *n* be the number of students in a class, where n is a multiple of 3. In each quiz, the students are divided into disjoint groups of exactly three. This means every student belongs to exactly one group per quiz.

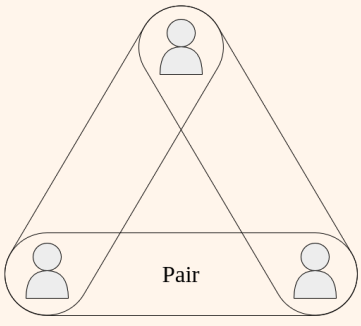


Fig. 1: Group of 3 Students

Let *n* be the number of students in a class, where *n* is divisible by 3. In each quiz, the students are divided into groups of exactly three. This means every student appears in exactly one group per quiz. Our goal is to conduct as many quizzes as possible under the constraint that **a pair of students does not repeat itself; no two students are ever grouped together more than once** throughout the entire series.

More formally, let *S* be a set of *n* students. Each quiz represents a partition of *S* into 3 element subsets. Each of these subsets contains three unique pairs of students as shown in Fig. 1.

Our constraint is that the union of all pairs across all quizzes must contain no duplicates. The core challenge is twofold:

1. **The Optimization Problem:** What is the maximum number of quizzes possible for a given *n*?
2. **The Construction Problem:** How can we actually build these groupings efficiently?

Historical Context

This problem is closely related to a classical work in combinatorial design theory, the **Steiner Triple System (STS)**. A Steiner Triple System of order *n*, denoted *STS(n)*, is a collection of 3 element subsets such that every pair of elements appears in exactly one subset. The roots of this concept date back to 19th century when Thomas Kirkman proposed his famous **"Kirkman's Schoolgirl Problem"**.

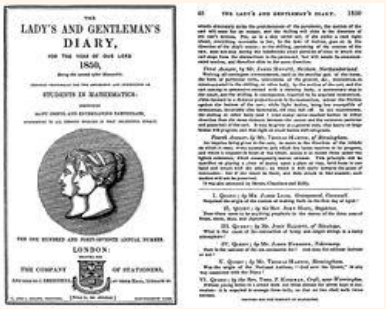

LADY'S AND GENTLEMAN'S
DIARY,
FOR THE YEAR OF OUR LORD
1850,
STUDENTS IN MATHEMATICS.
LONDON:
THE COMPANY OF STATIONERS,

Fig. 2: Original Publication of Kirkman's Schoolgirl Problem

> "Fifteen young ladies in a school walk out three abreast for seven days in succession: it is required to arrange them daily, so that no two shall walk twice abreast."

This is a specific instance of our problem where *n*=15.

Optimization Problem: Theoretical Upper Bound

Counting Total Pair Capacity

In a class of *n* students, the total number of unique pairs that can be formed is:

$$^{n}C_{2} = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$$

Counting Pairs Utilized per Quiz

In the introduction we assumed that *n* is divisible by 3 and the group size is 3, therefore **each quiz consists of $\frac{n}{3}$ groups**. From Fig. 1 we can observe that each group of three students contains exactly 3 unique pairs. The total number of pairs used in one quiz becomes,

$$3 \times \frac{n}{3} = n$$

Calculating the Limit

Because no pair can be repeated, the number of quizzes is strictly limited by the total number of pairs available divided by the pairs consumed per quiz:

$$Max\ Quizzes = \frac{\frac{n(n-1)}{2}}{n} = \frac{(n-1)}{2}$$

The Integrity Constraint

In a real-world setting, we cannot conduct a fraction of a quiz. Therefore, if $(n - 1)/2$ is not a whole number, we must take the **floor** (rounding down):

$$Max\ Quizzes = \lfloor \frac{(n-1)}{2} \rfloor$$

Formal Proof of Upper Bound

We want to prove that $\lfloor \frac{(n-1)}{2} \rfloor$ is the upper bound.

As derived while counting total pair capacity, the total number of unique pairs available in a set of *n* elements is $\frac{n(n-1)}{2}$. Each quiz consumes exactly *n* unique pairs, as seen while counting pairs utilized per quiz.

If we let *Q* be the number of quizzes, the total number of pairs used is $Q \times n$. Because we cannot use more pairs than exist in the set, we have:

$$Q \times n \leq \frac{n(n-1)}{2}$$

Dividing by *n:*

$$Q \leq \frac{(n-1)}{2}$$

Since *Q* must be an integer, $Q_{max} = \lfloor \frac{(n-1)}{2} \rfloor$. This proves that **no construction can ever exceed this value**.

It is important to emphasize that the proof above establishes only a theoretical upper bound, showing that the number of quizzes can never exceed $\lfloor \frac{(n-1)}{2} \rfloor$. However, this result is purely existential in nature: it does not, by itself, guarantee that such a configuration can always be realized for an arbitrary value of *n*. In the world of combinatorial design, proving that a solution is “not impossible” is only the first step; the real challenge lies in explicitly constructing such a configuration and understanding the structure that makes it attainable.

Construction Problem

Existing Work

Before moving forward, it is important to look at how this problem is currently handled. Current solutions generally fall into two categories: **Mathematical** and **Computational**.

Mathematical

Most theoretical solutions are based on **Steiner Triple Systems** and their variants, such as **Kirkman Triple Systems**. These methods are primarily *existence based* meaning they prove that a valid grouping is possible without necessarily providing an explicit procedure for constructing it in practice. Common techniques include:

- **Finite Fields and Projective Geometry:** Students are modeled as points in finite geometric spaces, where valid triples correspond to parallel lines. This structure guarantees that no pair of students appears together more than once.

- **Difference Sets:** A small base triple is defined using modular arithmetic, which is then shifted or rotated to generate all required groupings.

While mathematically elegant, these approaches are often "black boxes" for practical application. They typically require n to follow strict divisibility rules (like $n \equiv 3 \pmod 6$) and are presented as proofs of existence rather than step-by-step algorithms that a developer can easily adapt for arbitrary, non-ideal class sizes.

Computational

In computer science, this is often called the **Social Golfer Problem**. Instead of using formulas, computers use:

- **SAT Solvers & Brute Force:** The computer tries every possible combination until it finds one that works.
- **Constraint Programming:** The computer follows a set of rules and "backtracks" whenever it hits a mistake.

The Downside: These are "opaque" solutions. The computer might give you a valid schedule, but it can’t explain the logic of *why* those students were grouped that way. Furthermore, as the class size grows, the time it takes to find a solution can explode, making it inefficient for larger sets.

Most solutions come from combinatorial design theory, specifically Steiner and Kirkman Triple Systems. While these prove that groupings exist, they often rely on algebraic concepts like finite fields or difference sets. To a programmer, these are "existence-based" rather than "instruction-based", they prove a solution is possible but don't offer a simple, step-by-step recipe to build it. These approaches are mainly existence based and rely on algebraic or recursive constructions using finite fields, difference sets, or known base designs that are expanded to larger sizes. While these methods prove that valid groupings exist for certain values of n, they are often non-intuitive, mathematically heavy, and not designed as step-by-step algorithms that can be easily implemented or adapted. On the computational side, the problem has been studied as a special case of the Social Golfer Problem, where solutions are found using constraint programming, SAT solvers, or brute-force search with pruning. These methods work well for small or medium values of n, but they are computationally expensive and do not provide a simple deterministic construction. As a result, despite extensive theoretical and computational work, there is still a gap between existence proofs and a clean, constructive, algorithmic method that can be easily explained, implemented, and used in practical settings such as classrooms.